



Spring DATA JPA

- ♦ CRUD Operations called Insert, Update, Retrive and Delete Opearitions are common across all DAOs. The only difference is domain types you are interacting with.
- ♦ Spring Data internally provides the implementation for all such common methods. So developer is not responsible for writing implementation for these common methods.

Ex:

```
public interface CustomerDAO extends JpaRepository<Customer, Integer>{  
    //Empty  
}
```

- ♦ JpaRepository is parameterized and takes two parameters
 - Entity class
 - Primary Key type
- ♦ CrudRepository is the root for All the Dpring Data Repositories.
- ♦ CrudRepository has 12 Methods.
- ♦ PagingAndSortingRepository extends CrudRepository
- ♦ PagingAndSortingRepository has 2 Methods.
- ♦ JpaRepository extends PagingAndSortingRepository.
- ♦ JpaRepository has 12 methods and 14 methods will be inherited.
- ♦ Your DAO interface which is extending JpaRepository can use all these methods for performing common persistence operations such as saving, deleting, finding

Methods of PagingAndSortingRepository

1.	Page<T> findAll(Pageable pageable)
2.	Iterable<T> findAll(Sort sort)



Methods of CrudRepository

1.	long count()
2.	void delete(T entity)
3.	void deleteAll()
4.	void deleteAll(Iterable<? extends T> entities)
5.	void deleteAllById(Iterable<? extends ID> ids)
6.	void deleteById(ID id)
7.	boolean existsById(ID id)
8.	Iterable<T> findAll()
9.	Iterable<T> findAllById(Iterable<ID> ids)
10.	Optional<T> findById(ID id)
11.	S save(S entity)
12.	Iterable<S> saveAll(Iterable<S> entities)

Methods of JpaRepository

1.	void deleteAllByIdInBatch(Iterable<ID> ids)
2.	void deleteAllInBatch()
3.	void deleteAllInBatch(Iterable<T> entities)
4.	List<T> findAll()
5.	List<T> findAll(Sort sort)
6.	List<T> findAllById(Iterable<ID> ids)
7.	void flush()
8.	T getById(ID id)
9.	T getOne(ID id)
10.	List<S> saveAll(Iterable<S> entities)
11.	List<S> saveAllAndFlush(Iterable<S> entities)
12.	S saveAndFlush(S entity)



demo1-data-jpa: Files required

1. MyBootApplication.java	
2. Customer.java	
3. CustomerDAO.java	
4. application.properties	
5. pom.xml	

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        System.out.println(custDAO.getClass());

        // 1.save() method - inserting
        Customer cust1 = new Customer("bbb", "ccc@jlc", 222, "Blore");
        custDAO.save(cust1);
        System.out.println(cust1.getCid());

        //2. saveAndFlush() - A inserts if P.K not given
        Customer cust2 = new Customer("ccc", "ccc@jlc", 333, "Blore");
        custDAO.saveAndFlush(cust2);
```



//3. saveAndFlush() - B) updates if P.K given and present

```
Customer cust3 = new Customer(11,"bbb", "bbb@jlc", 222, "Blore");  
custDAO.saveAndFlush(cust3);
```

//4. saveAndFlush() -C) Inserts if P.K given and not present

```
Customer cust4 = new Customer(99,"raheem", "raheem@jlc",1234567, "Hyd");  
custDAO.saveAndFlush(cust4);
```

//5. findAll

```
List<Customer> mylist1 = custDAO.findAll();  
mylist1.stream().forEach(System.out::println);
```

//6. findAllById

```
List<Integer> myIds1 = Arrays.asList(11,12,13,14);  
List<Customer> mylist2 = custDAO.findAllById(myIds1);  
mylist2.stream().forEach(System.out::println);
```

//7. count

```
long count = custDAO.count();  
System.out.println(count);
```

//8. saveAll

```
Customer cust5 = new Customer("pp", "pp@jlc", 222, "Blore");  
Customer cust6 = new Customer("qq", "qq@jlc", 222, "Blore");  
Customer cust7 = new Customer("rr", "rr@jlc", 222, "Blore");  
List<Customer> custList = Arrays.asList(cust5,cust6,cust7);  
custDAO.saveAll(custList);
```

//9. findById

```
Optional<Customer> myOps = custDAO.findById(77);  
myOps.ifPresent(System.out::println);
```

//10. getOne

```
Customer cust8 = custDAO.getOne(77);  
System.out.println(cust8);
```

//11. getById

```
Customer cust9 = custDAO.getById(77);  
System.out.println(cust9);
```

//12. existsById

```
System.out.println(custDAO.existsById(77));  
System.out.println(custDAO.existsById(77));
```



```
//13. delete(entity)
Optional<Customer> myOps1 = custDAO.findById(502);
myOps1.ifPresent(mycust -> custDAO.delete(mycust));

//13. delete(entity)
if(myOps.isPresent()) {
    Customer cust = myOps.get();
    custDAO.delete(cust);
}

//14. deleteById
custDAO.deleteById(505);

//15. deleteAll
Customer cust10 = custDAO.getById(501);
Customer cust11 = custDAO.getById(503);
Customer cust12 = custDAO.getById(504);

List<Customer> custList1 = Arrays.asList(cust10,cust11,cust12);
custDAO.deleteAll(custList1);

//16. deleteAllInBatch
custDAO.deleteAllInBatch(custList1);

//17. deleteAllById
List<Integer> myIds = Arrays.asList(10,11,12);
custDAO.deleteAllById(myIds);

//18. deleteAllByIdInBatch
custDAO.deleteAllByIdInBatch(myIds);

//19. deleteAll
custDAO.deleteAll();

//20. deleteAllInBatch
custDAO.deleteAllInBatch();

System.out.println("Done!!!");
}

public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



2) Customer.java

```
package com.jlcindia.springboot;

import javax.persistence.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Entity
@Table(name="mycustomers")
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="cid")
    private int cid;

    @Column(name="cname")
    private String cname;

    @Column(name="email")
    private String email;

    @Column(name="phone")
    private int phone;

    @Column(name="city")
    private String city;

    public Customer() {}
    public Customer(String cname, String email, int phone, String city) {
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }
    public Customer(int cid, String cname, String email, int phone, String city) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }
    //Setters and Getters
    //toString()
}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import org.springframework.data.jpa.repository.JpaRepository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO extends JpaRepository<Customer, Integer>{

}
```

4)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#JPA Properties

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

5)pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo1-data-jpa</artifactId>
<version>1.0</version>
<name>demo1-data-jpa</name>
<description>This is Data JPA Example</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Demo2-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo2
2. CustomerService.java	Newly added in Demo2
3. CustomerServiceImpl.java	Newly added in Demo2
4. Customer.java	Same as Demo1
5. CustomerDAO.java	Same as Demo1
6. application.properties	Same as Demo1
7. pom.xml	Same as Demo1

1) MyBootApplication.java

```
package com.jlcindia.springboot;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```




```
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerService customerService;

    @Override
    public void run(String... args) throws Exception {

        customerService.findCustomerById(77).ifPresent(System.out::println);

        customerService.findCustomerById(7).ifPresent(System.out::println);

        customerService.getCustomerById(7).ifPresent(System.out::println);

        customerService.getCustomerById(77).ifPresent(System.out::println);
    }

    public static void main(String[] args) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```

2) CustomerService.java

```
package com.jlcindia.springboot;

import java.util.Optional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public interface CustomerService {
    public Optional<Customer> findCustomerById(int cid);
    public Optional<Customer> getCustomerById(int cid);
}
```



3) CustomerServiceImpl.java

```
package com.jlcindia.springboot;

import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Service
@Transactional
public class CustomerServiceImpl implements CustomerService{

    @Autowired
    CustomerDAO customerDAO;

    @Override
    @Transactional(isolation = Isolation.READ_COMMITTED,propagation =
    Propagation.REQUIRED)
    public Optional<Customer> findCustomerById(int cid) {
        Optional<Customer> custOpts= customerDAO.findById(cid);

        return custOpts;
    }

    @Override
    @Transactional(isolation = Isolation.READ_COMMITTED,propagation =
    Propagation.REQUIRED)
    public Optional<Customer> getCustomerById(int cid) {

        Optional<Customer> custOpts=Optional.empty();
        try {
            Customer cust = customerDAO.getById(cid);
            cust.getCname();
            custOpts=Optional.of(cust);
        }catch(Exception ex) { }

        return custOpts;
    }
}
```



- ♦ Spring Data Jpa not only provides implementation for commonly used methods but also provides a way to add custom methods.
- ♦ Spring Data only provides the implementation for custom methods.
- ♦ Spring Data defines Domain-Specific Language (DSL) where persistence details are expressed in method signature.

Ex:

```
interface CustomerDAO extends JpaRepository<Customer, Integer>{  
    List<Customer> findCustomerByCname(String name);  
    List<Customer> getCustomerByEmail(String email);  
  
    List<Customer> readByPhone(long phone);  
  
    List<Customer> getCustomerByEmailAndPhone(String email,long phone);  
    List<Customer> findCustomerByEmailOrPhone(String email,long phone);  
}
```

- ♦ Repository methods are combination of the following 4 things.
 1. **Verb,**
 2. **Optional Subject,**
 3. **word By,**
 4. **Predicate.**

Ex1:

```
findCustomerByEmail(String email)  
find is verb  
Customer is subject  
By is word  
Email is predicate
```

Ex2:

```
findByEmail(String email)  
find is verb  
the subject isn't specified and is implied to be a Customer.  
By is word  
Email is predicate
```

Ex3:

```
readCustomerByEmailOrPhone(String email, long phone)  
read is verb  
Customer is subject  
By is word  
EmailOrPhone is predicate
```



- ♦ Spring Data allows four verbs in the method name:
 - **get**
 - **read**
 - **find**
 - **count**.
- ♦ **get**, **read**, and **find** verbs are synonymous; all three query for data and return object(s).

```
List<Customer> getCustomerByEmail(String email);  
List<Customer> findCustomerByEmail(String email);  
List<Customer> readCustomerByEmail(String email);
```

- ♦ The **count** verb returns a count of matching objects, rather than the objects themselves.

```
long countCustomerByEmail(String email);
```

- ♦ The predicate is the most interesting and important part of the method name.
- ♦ Predicate contains Entity Class Properties and Operators
- ♦ We can use any of the following operators in Predicate

- 1) **IsAfter, After, IsGreaterThan, GreaterThan**
- 2) **IsGreaterThanEqual, GreaterThanEqual**
- 3) **IsBefore, Before, IsLessThan, LessThan**
- 4) **IsLessThanEqual, LessThanEqual**
- 5) **IsBetween, Between**
- 6) **IsNull, Null**
- 7) **IsNotNull, NotNull**
- 8) **IsIn, In**
- 9) **IsNotIn, NotIn**
- 10) **IsStartingWith, StartingWith, StartsWith**
- 11) **IsEndingWith, EndingWith, EndsWith**
- 12) **IsContaining, Containing, Contains**
- 13) **IsLike, Like**
- 14) **IsNotLike, NotLike**
- 15) **IsTrue, True**
- 16) **IsFalse, False**
- 17) **Is, Equals**
- 18) **IsNot, Not**



- ♦ We can sort the results by adding OrderBy at the end of the method name.
- ♦ Sort the results in ascending order by the cname.

```
List<Customer> readByCityAscCname(String city);
```

- ♦ Sort the results in descending order by the cname.

```
List<Customer> readByCityDescCname(String city);
```

- ♦ Spring Data DSL has its limitations, and sometimes it isn't convenient or even possible to express the desired query in a method name.
- ♦ In this case, Spring Data provides @Query annotation to write query explicitly.
- ♦ if we followed the naming convention, the method name would be incredibly long. In such situation, @Query can be useful.
- ♦ The @Query annotation is handy for adding custom query methods to a Spring Data JPA-enabled interface.

demo3-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo3
2. Customer.java	Updated in Demo3
3. CustomerDAO.java	Updated in Demo3
4. application.properties	Same as Demo2
5. pom.xml	Same as Demo2
6. mycustomers table	Updated in Demo3

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {
```



```
@Autowired
CustomerDAO custDAO;

@Override
public void run(String... args) throws Exception {
/*

//Insert 9 Records
Customer cust1 = new Customer("Sri", "Sri@jlc", 111, "Blore",5000);
Customer cust2 = new Customer("sd", "sd@jlc", 222, "Blore",25000);
Customer cust3 = new Customer("ds", "ds@jlc", 333, "Hyd",15000);
Customer cust4 = new Customer("hello", "hello@jlc", 12345, "Blore",15000);
Customer cust5 = new Customer("Sri", "Sri@jlc", 12345, "Delhi",9000);
Customer cust6 = new Customer("sd", "sd@jlc", 12345, "Blore",18000);
Customer cust7 = new Customer("ds", "ds@jlc", 333, "Delhi",17000);
Customer cust8 = new Customer("Sri", "Sri@jlc", 12345, "Hyd",23000);
Customer cust9 = new Customer("Hai", "hai@cc", 12345, "Hyd",23000);

List<Customer> custList =
Arrays.asList(cust1,cust2,cust3,cust4,cust5,cust6,cust7,cust8,cust9);

custDAO.saveAll(custList);
*/

List<Customer> mylist=null;

//1. getCustomersByCname
mylist=custDAO.getCustomersByCname("Sri");
mylist.forEach(System.out::println);

//2. getCustomersByEmail
mylist=custDAO.getCustomersByEmail("hello@jlc");
mylist.forEach(System.out::println);

//3. readCustomersByPhone
mylist=custDAO.readCustomersByPhone(12345);
mylist.forEach(System.out::println);

//4. findCustomersByCity
mylist=custDAO.findCustomersByCity("Hyd");
mylist.forEach(System.out::println);

//5. getCustomersByEmailAndPhone
mylist=custDAO.getCustomersByEmailAndPhone("hello@jlc", 12345);
mylist.forEach(System.out::println);
```



```
//6. getCustomersByEmailAndPhone
mylist=custDAO.getCustomersByEmailOrPhone("hello@jlc", 12345);
mylist.forEach(System.out::println);

//7. getCustomersByCnameLike
mylist=custDAO.getCustomersByCnameLike("sr%");
mylist.forEach(System.out::println);

//8. getCustomersByCnameStartsWith
mylist=custDAO.getCustomersByCnameStartsWith("sd");
mylist.forEach(System.out::println);

//9. getCustomersByEmailEndsWith
mylist=custDAO.getCustomersByEmailEndsWith("@cc");
mylist.forEach(System.out::println);

//10. getCustomersByEmailContains
mylist=custDAO.getCustomersByCnameContains("jlc");
mylist.forEach(System.out::println);

//11. getCustomersByCityIn
mylist=custDAO.getCustomersByCityIn("Hyd","Delhi");
mylist.forEach(System.out::println);

//12. getCustomersByCityNotIn
mylist=custDAO.getCustomersByCityNotIn("Hyd","Delhi");
mylist.forEach(System.out::println);

//13. countCustomersByCity
long count=custDAO.countCustomersByCity("Hyd");
System.out.println(count);

//14. countCustomersByCityIn
count=custDAO.countCustomersByCityIn("Hyd","Delhi");
System.out.println(count);

//15. getCustomersByCityNotIn
count=custDAO.countCustomersByCityNotIn("Blore");
System.out.println(count);

//16. countCustomersByEmailNull
count=custDAO.countCustomersByEmailNull();
System.out.println(count);

//17. countCustomersByEmailNotNull
count=custDAO.countCustomersByEmailNotNull();
System.out.println(count);
```




```
//18. getCustomersByEmailNull
mylist=custDAO.getCustomersByEmailNull();
mylist.forEach(System.out::println);

//19. getCustomersByEmailNotNull
mylist=custDAO.getCustomersByEmailNotNull();
mylist.forEach(System.out::println);

//20. getCustomersByBalance
mylist=custDAO.getCustomersByBalance(15000);
mylist.forEach(System.out::println);

//21. getCustomersByBalanceIsBetween
mylist=custDAO.getCustomersByBalanceIsBetween(10000,20000);
mylist.forEach(System.out::println);

//22. getCustomersByBalanceIsGreaterThan
mylist=custDAO.getCustomersByBalanceIsGreaterThan(20000);
mylist.forEach(System.out::println);

//23. getCustomersByBalanceLessThan
mylist=custDAO.getCustomersByBalanceLessThan(10000);
mylist.forEach(System.out::println);

//24. getCustomersByBalanceGreaterThanOrEqualTo
mylist=custDAO.getCustomersByBalanceGreaterThanOrEqualTo(20000);
mylist.forEach(System.out::println);

//25. getCustomersByBalanceLessThanOrEqualTo
mylist=custDAO.getCustomersByBalanceLessThanOrEqualTo(10000);
mylist.forEach(System.out::println);

System.out.println("Done!!!");

}

public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```

2) Customer.java

```
package com.jlcindia.springboot;

import javax.persistence.*;
/*
```




* **@Author : Srinivas Dande**

* **@Company: Java Learning Center**

* */

@Entity

@Table(name="mycustomers")

public class Customer {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

@Column(name="cid")

private int cid;

@Column(name="cname")

private String cname;

@Column(name="email")

private String email;

@Column(name="phone")

private int phone;

@Column(name="city")

private String city;

@Column(name="balance")

private double balance;

public Customer() {}

public Customer(String cname, String email, int phone, String city,double balance) {

this.cname = cname;

this.email = email;

this.phone = phone;

this.city = city;

this.balance=balance;

}

public Customer(int cid, String cname, String email, int phone, String city,double balance) {

this.cid = cid;

this.cname = cname;

this.email = email;

this.phone = phone;

this.city = city;

this.balance=balance;

}

//Setters and Getters

//toString()

}



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO extends JpaRepository<Customer, Integer> {

    List<Customer> getCustomersByCname(String cname);
    List<Customer> getCustomersByEmail(String email);
    List<Customer> readCustomersByPhone(int phone);
    List<Customer> findCustomersByCity(String city);

    List<Customer> getCustomersByEmailAndPhone(String email, int phone);
    List<Customer> getCustomersByEmailOrPhone(String email, int phone);

    List<Customer> getCustomersByCnameLike(String cname);
    List<Customer> getCustomersByCnameStartsWith(String cname);
    List<Customer> getCustomersByEmailEndsWith(String email);
    List<Customer> getCustomersByCnameContains(String email);

    List<Customer> getCustomersByCityIn(String... cities);
    List<Customer> getCustomersByCityNotIn(String... cities);

    long countCustomersByCity(String city);
    long countCustomersByCityIn(String... cities);
    long countCustomersByCityNotIn(String... cities);
    long countCustomersByCnameLike(String cname);

    long countCustomersByEmailNull();
    long countCustomersByEmailNotNull();

    List<Customer> getCustomersByEmailNull();
    List<Customer> getCustomersByEmailNotNull();

    List<Customer> getCustomersByBalance(double bal);
    List<Customer> getCustomersByBalanceIsBetween(double bal1, double bal2);
    List<Customer> getCustomersByBalanceIsGreaterThan(double bal);

    List<Customer> getCustomersByBalanceLessThan(double bal);
    List<Customer> getCustomersByBalanceGreaterThanEqual(double bal);
    List<Customer> getCustomersByBalanceLessThanEqual(double bal);
}
```



6) mycustomers table

```
create table mycustomers(  
  cid int primary key,  
  cname char(10),  
  email char(10),  
  phone long,  
  city char(10) ,  
  balance double );
```

demo4-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo4
2. Customer.java	Same as Demo3
3. CustomerDAO.java	Updated in Demo4
4. application.properties	Same as Demo3
5. pom.xml	Same as Demo3
6. mycustomers table	Same as Demo3

1) MyBootApplication.java

```
package com.jlcindia.springboot;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.boot.CommandLineRunner;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
@SpringBootApplication  
public class MyBootApplication implements CommandLineRunner {  
  
    @Autowired  
    CustomerDAO custDAO;  
  
    @Override  
    public void run(String... args) throws Exception {  
  
        List<Customer> mylist=null;
```



```
//1. getCustomersNyEmailAndPhone1
mylist=custDAO.getCustomersNyEmailAndPhone1("hello@jlc",12345);
mylist.forEach(System.out::println);

//2. getCustomersNyEmailAndPhone2
mylist=custDAO.getCustomersNyEmailAndPhone2("hello@jlc",12345);
mylist.forEach(System.out::println);

//3. getCustomersNyEmailOrPhone1
mylist=custDAO.getCustomersNyEmailOrPhone1("hello@jlc",12345);
mylist.forEach(System.out::println);

//4. getCustomersNyEmailOrPhone2
mylist=custDAO.getCustomersNyEmailOrPhone2("hello@jlc",12345);
mylist.forEach(System.out::println);

//5. findCustomersByCity1
mylist=custDAO.findCustomersByCity1("Blore");
mylist.forEach(System.out::println);

//6. findCustomersByCity2
mylist=custDAO.findCustomersByCity2("Blore");
mylist.forEach(System.out::println);

//7. findAllCustomers1
mylist=custDAO.findAllCustomers1();
mylist.forEach(System.out::println);

//8. findAllCustomers2
mylist=custDAO.findAllCustomers2();
mylist.forEach(System.out::println);

//9. findCustomersByPhone1
mylist=custDAO.findCustomersByPhone1(12345);
mylist.forEach(System.out::println);

//10. findCustomersByPhone2
mylist=custDAO.findCustomersByPhone2(12345);
mylist.forEach(System.out::println);

System.out.println("Done!!!");
}
public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO extends JpaRepository<Customer, Integer> {

    @Query(value = "from Customer cust where email=?1 and phone=?2")
    List<Customer> getCustomersNyEmailAndPhone1(String email, int phone);

    @Query(value = "select * from mycustomers cust where email=?1 and phone=?2",
    nativeQuery = true)
    List<Customer> getCustomersNyEmailAndPhone2(String email, int phone);

    @Query(value = "from Customer cust where email=?1 or phone=?2")
    List<Customer> getCustomersNyEmailOrPhone1(String email, int phone);

    @Query(value = "select * from mycustomers cust where email=?1 or phone=?2",
    nativeQuery = true)
    List<Customer> getCustomersNyEmailOrPhone2(String email, int phone);

    @Query(value = "from Customer cust where city=?1 order by cname desc")
    List<Customer> findCustomersByCity1(String city);

    @Query(value = "select * from myustomers cust where city=?1 order by cname desc",
    nativeQuery = true)
    List<Customer> findCustomersByCity2(String city);

    @Query(value = "from Customer cust order by cname asc")
    List<Customer> findAllCustomers1();

    @Query(value = "select * from myustomers cust order by cname asc", nativeQuery = true)
    List<Customer> findAllCustomers2();

    @Query(value = "from Customer cust where cust.phone = :myphone")
    List<Customer> findCustomersByPhone1(@Param("myphone") long phone);

    @Query(value = "select * from customers where cust.phone = :myphone", nativeQuery
    = true)
    List<Customer> findCustomersByPhone2(@Param("myphone") long phone);
}
```



demo5-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo5
2. Customer.java	Same as Demo4
3. CustomerDAO.java	Updated in Demo5
4. application.properties	Same as Demo4
5. pom.xml	Same as Demo4
6. mycustomers table	Same as Demo4

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.Sort.Order;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        //1. getAllCustomers - phoneSort
        Sort phoneSort=Sort.by(Order.asc("phone"));
        mylist=custDAO.getAllCustomers(phoneSort);
        mylist.forEach(System.out::println);
    }
}
```



```
//1. getAllCustomers - nameSort
Sort nameSort=Sort.by(Order.asc("cname"));
mylist=custDAO. getAllCustomers(nameSort);
mylist.forEach(System.out::println);

//2. getCustomersByCity - phoneSort
mylist=custDAO. getCustomersByCity("Blore",phoneSort);
mylist.forEach(System.out::println);

//2. getCustomersByCity - nameSort
mylist=custDAO. getCustomersByCity("Blore",nameSort);
mylist.forEach(System.out::println);

//3. getAllCustomers - mypage1
Pageable mypage1=PageRequest.of(0, 5);
mylist=custDAO. getAllCustomers(mypage1);
mylist.forEach(System.out::println);

//3. getAllCustomers - mypage2
Pageable mypage2=PageRequest.of(0, 5,Sort.by(Order.asc("cname")));
mylist=custDAO. getAllCustomers(mypage2);
mylist.forEach(System.out::println);

//4. getCustomersByCity - mypage3
Pageable mypage3=PageRequest.of(0, 3);
mylist=custDAO.getCustomersByCity("Blore",mypage3);
mylist.forEach(System.out::println);

//4. getCustomersByCity - mypage4
Pageable mypage4=PageRequest.of(0, 3,Sort.by(Order.asc("phone")));
mylist=custDAO. getCustomersByCity("Blore",mypage4);
mylist.forEach(System.out::println);

System.out.println("Done!!!");
}

public static void main(String[] args) {
SpringApplication.run(MyBootApplication.class, args);
}

}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public interface CustomerDAO extends JpaRepository<Customer, Integer> {

    @Query("from Customer cust ")
    List<Customer> getAllCustomers(Sort sort);

    @Query("from Customer cust where city=?1")
    List<Customer> getCustomersByCity(String city,Sort sort);

    @Query("from Customer cust")
    List<Customer> getAllCustomers(Pageable pageable);

    @Query("from Customer cust where city=?1")
    List<Customer> getCustomersByCity(String city,Pageable pageable);

}
```




Spring DATA Mongo

- ♦ CRUD Operations called Insert, Update, Retrive and Delete Opearions are common across all DAOs. The only difference is domain types you are interacting with.
- ♦ Spring Data internally provides the implementation for all such common methods. So developer is not responsible for writing implementation for these common methods.

Ex:

```
public interface CustomerDAO extends MongoRepository<Customer,  
String>{  
    //Empty  
}
```

- ♦ **MongoRepository** is parameterized and takes two parameters
 - Domain class
 - Object Id Type
- ♦ **CrudRepository** is the root for All the Dpring Data Repositories.
- ♦ CrudRepository has 12 Methods.
- ♦ **PagingAndSortingRepository** extends CrudRepository
- ♦ PagingAndSortingRepository has 2 Methods.
- ♦ **MongoRepository** extends PagingAndSortingRepository.
- ♦ **MongoRepository** has 2 methods extra and 14 methods will be inherited.
- ♦ Your DAO interface which is extending **MongoRepository** can use all these methods for performing common persistence operations such as saving, deleting, finding

Methods of PagingAndSortingRepository

1.	Page<T> findAll(Pageable pageable)
2.	Iterable<T> findAll(Sort sort)



Methods of CrudRepository

1.	long count()
2.	void delete(T entity)
3.	void deleteAll()
4.	void deleteAll(Iterable<? extends T> entities)
5.	void deleteAllById(Iterable<? extends ID> ids)
6.	void deleteById(ID id)
7.	boolean existsById(ID id)
8.	Iterable<T> findAll()
9.	Iterable<T> findAllById(Iterable<ID> ids)
10.	Optional<T> findById(ID id)
11.	S save(S entity)
12.	Iterable<S> saveAll(Iterable<S> entities)

Methods of JpaRepository

1.	S insert(S entity)
2.	List<S> insert(Iterable<S> entities)

demo6-data-mongo: Working Steps

A) Setup MongoDB

- 1) Make sure that MongoDB is installed on your machine
- 2) Open MongoDB Compass and connect with the following URL:

mongodb://localhost:27017/

- 3) Create the database called **myjlcdb5**
- 4) Create the collection called **mycustomers**
- 5) Insert some **5 Documents** in **mycustomers** collection

```
[ {  
  "cid": 101,  
  "cname": "Sri",  
  "email": "sri@jlc",  
  "phone": 111,  
  "city": "Blore",  
  "balance": 15000  
},
```



```
{
  "cid": 102,
  "cname": "Hello",
  "email": "hello@jlc",
  "phone": 333,
  "city": "Blore",
  "balance": 25000
},
{
  "cid": 103,
  "cname": "SD",
  "email": "sd@jlc",
  "phone": 444,
  "city": "Hyd",
  "balance": 18000
},
{
  "cid": 104,
  "cname": "DS",
  "email": "ds@jlc",
  "phone": 555,
  "city": "Hyd",
  "balance": 8000
},
{
  "cid": 105,
  "cname": "Hai",
  "email": "hai@jlc",
  "phone": 222,
  "city": "Blore",
  "balance": 12000
}
]
```

B) Create Spring Boot Project

6) Copy demo1-data-jpa project and paste as demo6-data-mongo

7) Update **pom.xml**

```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
</dependencies>
```



8) Update **application.properties**

```
# MongoDB Properties
#spring.data.mongodb.authentication-database=admin
#spring.data.mongodb.username=srinivas
#spring.data.mongodb.password=srinivas
spring.data.mongodb.database=myjlcdb5
spring.data.mongodb.port=27017
spring.data.mongodb.host=localhost

#Log Properties
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

9) Update **Customer.java**

```
@Document(collection = "mycustomers")
public class Customer {

    @Id
    private String myobjectId;

    private String cid;
    private String cname;
    private String email;
    private int phone;
    private String city;
    private double balance;

    //Constructors
    //Setters and Getters
    //toString()

}
```

10)Update **CustomerDAO.java**

```
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String> {

}
```

11)Update **MyBootApplication.java**

⇒ You can call and test methods from **MongoRepository**



demo6-data-mongo: Files required

1. MyBootApplication.java	
2. Customer.java	
3. CustomerDAO.java	
4. application.properties	
5. pom.xml	

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.Sort.Order;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        // 1.save() method - inserting
        Customer cust1 = new Customer("106", "vas", "vas@jlc", 12345, "Blore", 7500);
        custDAO.save(cust1);

        // 2.insert() method - inserting
        Customer cust2 = new Customer("108","ppp", "ppp@jlc", 8888, "Blore",12500);
        custDAO.insert(cust2);
```



// 3.save() method - Updating

```
Optional<Customer> myOps1 = custDAO.findById("611d0d8234971e6cc53f6791");
if(myOps1.isPresent()) {
    Customer cust = myOps1.get();
    cust.setCid("107");
    cust.setCname("ddd");
    cust.setEmail("ddd@jlc");
    cust.setPhone(55555);
    custDAO.save(cust);
}
```

//4. existsById

```
boolean b= custDAO.existsById("611cd83bd22f1e5adfbf6a95");
System.out.println("Document Found : "+ b);
```

//5. findById

```
Optional<Customer> myOps2 = custDAO.findById("611cd83bd22f1e5adfbf6a95");
myOps2.ifPresent(System.out::println);
```

//6. deleteById

```
custDAO.deleteById("611d00e1cd9ff57d738acc08");
```

//7. delete

```
Optional<Customer> myOps3 = custDAO.findById("611d02cd739d2c69846025e8");
myOps3.ifPresent(mycust -> custDAO.delete(mycust));
```

//8. count

```
System.out.println("Total Customer : "+ custDAO.count());
```

//9. findAll()

```
mylist=custDAO.findAll();
mylist.forEach(System.out::println);
```

//10. findAll(Sort)

```
Sort nameSort = Sort.by(Order.asc("cname"));
mylist = custDAO.findAll(nameSort);
mylist.stream().forEach(System.out::println);
```

//You can test the remaining methods

```
System.out.println("Done!!!");
}
public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



2) Customer.java

```
package com.jlcindia.springboot;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.index.Indexed;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@Document(collection = "mycustomers")
public class Customer {

    @Id
    private String myobjectId;

    @Indexed
    private String cid;

    @Indexed
    private String cname;
    private String email;
    private int phone;
    private String city;
    private double balance;

    public Customer() {}
    public Customer(String cid, String cname, String email, int phone, String city,double balance)
    {
        this.cid = cid;
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
        this.balance=balance;
    }

    //Setters and Getters
    //toString()

}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String> {

}
```

4)application.properties

MongoDB Properties

```
#spring.data.mongodb.authentication-database=admin
#spring.data.mongodb.username=srinivas
#spring.data.mongodb.password=srinivas
spring.data.mongodb.database=myjlcdb5
spring.data.mongodb.port=27017
spring.data.mongodb.host=localhost
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

5)pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo6-data-mongo</artifactId>
<version>1.0</version>
<name>demo6-data-mongo</name>
<description>This is Spring Boot- MongoDB Example</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
<relativePath/>
</parent>
```




```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



demo7-data-mongo: Files required

1. MyBootApplication.java	Updated in Demo7
2. Customer.java	Same as Demo6
3. CustomerDAO.java	Updated in Demo7
4. application.properties	Same as Demo6
5. pom.xml	Same as Demo6

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        //1. getCustomersByCname
        mylist=custDAO.getCustomersByCname("Sri");
        mylist.forEach(System.out::println);
        System.out.println("-----");

        //2. getCustomersByEmail
        mylist=custDAO.getCustomersByEmail("hello@jlc");
        mylist.forEach(System.out::println);
        System.out.println("-----");
    }
}
```



//3. readCustomersByPhone

```
mylist=custDAO.readCustomersByPhone(12345);  
mylist.forEach(System.out::println);  
System.out.println("-----");
```

//4. findCustomersByCity

```
mylist=custDAO.findCustomersByCity("Hyd");  
mylist.forEach(System.out::println);  
System.out.println("-----");
```

//5. getCustomersByEmailAndPhone

```
mylist=custDAO.getCustomersByEmailAndPhone("hello@jlc", 12345);  
mylist.forEach(System.out::println);  
System.out.println("-----");
```

//6. getCustomersByEmailOrPhone

```
mylist=custDAO.getCustomersByEmailOrPhone("hello@jlc", 12345);  
mylist.forEach(System.out::println);  
System.out.println("-----");
```

//7. getCustomersByCityIn

```
mylist=custDAO.getCustomersByCityIn("Hyd","Delhi");  
mylist.forEach(System.out::println);  
System.out.println("-----");
```

//8. getCustomersByCityNotIn

```
mylist=custDAO.getCustomersByCityNotIn("Hyd","Delhi");  
mylist.forEach(System.out::println);
```

//9. countCustomersByCity

```
long count=custDAO.countCustomersByCity("Hyd");  
System.out.println(count);  
System.out.println("-----");
```

//10. countCustomersByCityIn

```
count=custDAO.countCustomersByCityIn("Hyd","Delhi");  
System.out.println(count);  
System.out.println("-----");
```

//11. getCustomersByCityNotIn

```
count=custDAO.countCustomersByCityNotIn("Blore");  
System.out.println(count);  
System.out.println("-----");
```



```
//12. getCustomersByBalance
mylist=custDAO.getCustomersByBalance(15000);
mylist.forEach(System.out::println);
System.out.println("-----");

//13. getCustomersByBalanceIsBetween
mylist=custDAO.getCustomersByBalanceIsBetween(10000,20000);
mylist.forEach(System.out::println);
System.out.println("-----");

//14. getCustomersByBalanceIsGreaterThan
mylist=custDAO.getCustomersByBalanceIsGreaterThan(20000);
mylist.forEach(System.out::println);
System.out.println("-----");

//15. getCustomersByBalanceLessThan
mylist=custDAO.getCustomersByBalanceLessThan(10000);
mylist.forEach(System.out::println);
System.out.println("-----");

//16. getCustomersByBalanceGreaterThanOrEqualTo
mylist=custDAO.getCustomersByBalanceGreaterThanOrEqualTo(20000);
mylist.forEach(System.out::println);
System.out.println("-----");

//17. getCustomersByBalanceLessThanOrEqualTo
mylist=custDAO.getCustomersByBalanceLessThanOrEqualTo(10000);
mylist.forEach(System.out::println);
System.out.println("-----");

System.out.println("Done!!!");

}
public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String> {

    List<Customer> getCustomersByCname(String cname);
    List<Customer> getCustomersByEmail(String email);
    List<Customer> readCustomersByPhone(int phone);
    List<Customer> findCustomersByCity(String city);

    List<Customer> getCustomersByEmailAndPhone(String email, int phone);
    List<Customer> getCustomersByEmailOrPhone(String email, int phone);

    List<Customer> getCustomersByCityIn(String... cities);
    List<Customer> getCustomersByCityNotIn(String... cities);

    long countCustomersByCity(String city);
    long countCustomersByCityIn(String... cities);
    long countCustomersByCityNotIn(String... cities);

    List<Customer> getCustomersByBalance(double bal);
    List<Customer> getCustomersByBalanceIsBetween(double bal1, double bal2);

    List<Customer> getCustomersByBalanceIsGreaterThan(double bal);
    List<Customer> getCustomersByBalanceLessThan(double bal);

    List<Customer> getCustomersByBalanceGreaterThanEqual(double bal);
    List<Customer> getCustomersByBalanceLessThanEqual(double bal);
}
```



demo8-data-mongo: Files required

1. MyBootApplication.java	Updated in Demo8
2. Customer.java	Same as Demo7
3. CustomerDAO.java	Updated in Demo8
4. application.properties	Same as Demo7
5. pom.xml	Same as Demo7

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        //1. fetchCustomersByCity
        mylist=custDAO.fetchCustomersByCity("Hyd");
        mylist.forEach(System.out::println);
        System.out.println("-----");

        //2. fetchCustomersByPhone
        mylist=custDAO.fetchCustomersByPhone(12345);
        mylist.forEach(System.out::println);

        //3. fetchCustomersByBal
        mylist=custDAO.fetchCustomersByBal(15000);
        mylist.forEach(System.out::println);
        System.out.println("-----");
    }
}
```



```
//4. fetchCustomersByBalGTE
mylist=custDAO.fetchCustomersByBalGTE(15000);
mylist.forEach(System.out::println);
System.out.println("-----");

//5. fetchCustomersByBalLTE
mylist=custDAO.fetchCustomersByBalLTE(15000);
mylist.forEach(System.out::println);
System.out.println("-----");

//6. fetchCustomersByBalRange
mylist=custDAO.fetchCustomersByBalRange(10000,20000);
mylist.forEach(System.out::println);
System.out.println("-----");

//7. fetchCustomersByEmailAndPhone
mylist=custDAO.fetchCustomersByEmailAndPhone("hello@jlc",12345);
mylist.forEach(System.out::println);
System.out.println("-----");

//8. fetchCustomersByEmailOrPhone
mylist=custDAO.fetchCustomersByEmailOrPhone("hello@jlc",12345);
mylist.forEach(System.out::println);
System.out.println("-----");

//9. fetchCustomersByCitiesIN
mylist=custDAO.fetchCustomersByCitiesIN("Hyd","Delhi");
mylist.forEach(System.out::println);
System.out.println("-----");

//10. fetchCustomersByCitiesNIN
mylist=custDAO.fetchCustomersByCitiesNIN("Hyd","Delhi");
mylist.forEach(System.out::println);
System.out.println("-----");

System.out.println("Done!!!");

}

public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}

}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.data.mongodb.repository.Query;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String> {

    @Query("{ 'city':?0}")
    List<Customer> fetchCustomersByCity(String mycity);

    @Query("{ 'phone':?0}")
    List<Customer> fetchCustomersByPhone(int myphone);

    @Query("{ 'balance':?0}")
    List<Customer> fetchCustomersByBal(double bal);

    @Query("{ 'balance': {$gte:?0}}")
    List<Customer> fetchCustomersByBalGTE(double bal);

    @Query("{ 'balance': {$lte:?0}}")
    List<Customer> fetchCustomersByBalLTE(double bal);

    @Query("{ $and : [ { 'balance': {$gte:?0}} , { 'balance': {$lte:?1}} ]}")
    List<Customer> fetchCustomersByBalRange(double bal1, double bal2);

    @Query("{ $and : [ { 'email': ?0} , { 'phone':?1} ]}")
    List<Customer> fetchCustomersByEmailAndPhone(String email, int phone);

    @Query("{ $or : [ { 'email': ?0} , { 'phone':?1} ]}")
    List<Customer> fetchCustomersByEmailOrPhone(String email, int phone);

    @Query("{ 'city':{$in:?0}}")
    List<Customer> fetchCustomersByCitiesIN(String... mycities);

    @Query("{ 'city':{$nin:?0}}")
    List<Customer> fetchCustomersByCitiesNIN(String... mycities);
}
```




demo9-data-mongo: Files required

1. MyBootApplication.java	Updated in Demo9
2. Customer.java	Same as Demo8
3. CustomerDAO.java	Updated in Demo9
4. application.properties	Same as Demo8
5. pom.xml	Same as Demo8

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.Sort.Order;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        //1. findAll - phoneSort
        Sort phoneSort1 = Sort.by(Order.desc("phone"));

        mylist=custDAO.findAll(phoneSort1);
        mylist.forEach(System.out::println);

        System.out.println("-----");
    }
}
```



//1. findAll - emailSort

```
Sort emailSort1 = Sort.by(Order.asc("email"));
```

```
mylist=custDAO.findAll(emailSort1);  
mylist.forEach(System.out::println);  
System.out.println("-----");
```

//2. findAll - mypage1

```
Page<Customer> pageList=null;  
Pageable mypage1= PageRequest.of(0, 5);  
pageList=custDAO.findAll(mypage1);
```

```
pageList.forEach(System.out::println);  
System.out.println("-----");
```

//2. findAll - mypage2

```
Sort emailSort = Sort.by(Order.asc("email"));  
Pageable mypage2= PageRequest.of(0, 5,emailSort);  
pageList=custDAO.findAll(mypage2);  
pageList.forEach(System.out::println);  
System.out.println("-----");
```

//3. fetchCustomersByCity - emailSort

```
mylist=custDAO.fetchCustomersByCity("Blore",emailSort);  
mylist.forEach(System.out::println);
```

//4. fetchCustomersByCity - mypage3

```
List<Customer> custList=null;  
Pageable mypage3= PageRequest.of(0, 3);  
custList=custDAO.fetchCustomersByCity("Blore",mypage3);  
custList.forEach(System.out::println);  
System.out.println("-----");
```

//4. fetchCustomersByCity - mypage4

```
Pageable mypage4= PageRequest.of(0, 3,emailSort);  
custList=custDAO.fetchCustomersByCity("Blore",mypage4);  
custList.forEach(System.out::println);  
System.out.println("-----");
```

```
System.out.println("Done!!!");  
}
```

```
public static void main(String[] args) {  
SpringApplication.run(MyBootApplication.class, args);  
}  
}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.data.mongodb.repository.Query;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String>{

    @Query("{ 'city' : ?0}")
    List<Customer> fetchCustomersByCity(String city,Sort sort);

    @Query("{ 'city' : ?0}")
    List<Customer> fetchCustomersByCity(String city,Pageable pageable);

}
```

demo10-data-mongo: Files required

1. MyBootApplication.java	Updated in Demo10
2. Customer.java	Same as Demo9
3. CustomerDAO.java	Updated in Demo10
4. CustomerDAOImpl.java	Newly Added in Demo10
5. application.properties	Same as Demo9
6. pom.xml	Same as Demo9

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```



```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.save()
        Customer cust = new Customer("109", "asd", "asd@jlc", 12345, "Blore", 17500);
        custDAO.addCustomer(cust);

        //2. getAllCustomers
        List<Customer> mylist=custDAO.getAllCustomers();
        mylist.forEach(System.out::println);

        System.out.println("Done!!!");
    }

    public static void main(String[] args) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```

3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.stereotype.Repository;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public interface CustomerDAO {
    public void addCustomer(Customer cust);
    public List<Customer> getAllCustomers();
}
```



4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public class CustomerDAOImpl implements CustomerDAO {

    @Autowired
    MongoTemplate mongoTemp;

    @Override
    public void addCustomer(Customer cust) {
mongoTemp.save(cust);
    }

    @Override
    public List<Customer> getAllCustomers() {
return mongoTemp.findAll(Customer.class);
    }

}
```